

Atividade Autônoma 4 - Testes e Cobertura de Testes

Vocês vão trabalhar em cima do código da Atividade Autônoma 3

1. Atualizar o pom.xml (vai estar no final das explicações desse documento)

2. Objetivos do Aprendizado

- Refatorar código acoplado para torná-lo testável (aplicando inversão de controle/injeção de dependência).
- Escrever casos de teste unitários abrangentes utilizando o **JUnit 5**.
- Analisar e atingir metas de cobertura de código em branches lógicas usando o **JaCoCo**.

3. Cenário e Contexto

O sistema atual analisa a elegibilidade baseando-se em valores fixos no código (`media > 8.5` e `faltas < 3`). Agora, o sistema deve se tornar dinâmico.

As regras de aprovação variam de acordo com o **Tipo de Bolsa** e devem ser carregadas a partir de um arquivo externo `regras_bolsa.csv` localizado na máquina do usuário. Além disso, o formulário da aplicação recebeu novos campos: **Tipo de Bolsa** e **Renda Familiar**.

4. Atividade Passo a Passo

Passo 1: O Arquivo de Configuração Local

Crie um arquivo chamado `regras_bolsa.csv` na raiz do seu projeto (ou em um caminho absoluto configurado na sua aplicação) com o seguinte conteúdo:

```
tipoBolsa,mediaMinima,limiteFaltas,exigeRendaBaixa,rendaMaxima
IniciacaoCientifica,8.5,3,false,0.0
Extensao,7.5,4,false,0.0
AuxilioPermanencia,7.0,5,true,2500.0
```

Passo 2: Refatoração Orientada a Testabilidade

Se a sua classe `AnaliseElegibilidade` abrir e ler o arquivo CSV de dentro do método de cálculo, **você não conseguirá testar as regras de negócio de forma isolada**, pois o teste falhará se o arquivo sumir ou mudar.

Sua missão:

1. Crie uma classe (ou registro) `RegraBolsa` para representar uma linha do CSV.
2. Crie uma classe utilitária encarregada exclusivamente de ler o CSV e retornar uma lista ou mapa de regras.
3. Altere o método da classe `AnaliseElegibilidade` para receber a regra correspondente e os dados do aluno.

Passo 3: Criação dos Testes Unitários (JUnit 5)

Crie uma classe de teste em

`src/test/java/br/edu/ufrgs/model/AnaliseElegibilidadeTest.java`

Escreva testes unitários para validar os diferentes caminhos lógicos (cenários de sucesso e falhas para cada tipo de bolsa).

Exemplo com um teste só! Vocês precisam criar os demais!!!

```
package br.edu.ufrgs.model;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class AnaliseElegibilidadeTest {

    @Test

    public void testIniciacaoCientificaComSucesso() {

        AnaliseElegibilidade analise = new AnaliseElegibilidade();

        // Criando um cenário controlado sem precisar ler o
        arquivo físico do disco.

        RegraBolsa regraIC = new RegraBolsa("IniciacaoCientifica",
            8.5, 3, false, 0.0);

        String resultado = analise.verificaElegibilidadeBolsa(9.0,
            2, 0.0, regraIC);

        assertEquals("Candidato Elegível", resultado);

    }

    // TODO: Escreva testes para as demais ramificações (IFs) do
    código!

}
```

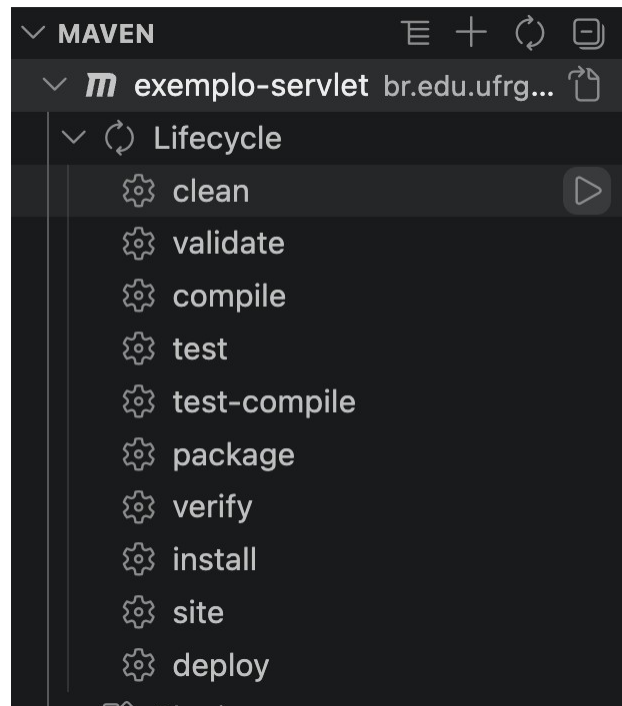
Não esqueça dos testes de leitura do csv. Você vai precisar de um RegraBolsaTest.java

Passo 4: Executando os Testes e Gerando o Relatório JaCoCo

Use a integração da sua IDE favorita. No VSCode, procure pelo Maven, à esquerda expanda a opção Lifecycle

Execute os comandos clean, test-compile, test

Observe a saída do terminal para cada comando executado. O terminal vai mostrar quais testes passaram e quais não.



Passo 5: Analisando a Cobertura de Código

Após a execução do comando, navegue até a pasta do seu projeto e abra o seguinte arquivo no navegador: <target/site/jacoco/index.html>

O relatório mostrará graficamente quais linhas e ramificações (branches) do seu código foram exercitadas pelos testes (verde) e quais ficaram de fora (vermelho).

5. Critérios de Entrega e Avaliação

- Código sem Acoplamento I/O: A lógica de validação não pode depender diretamente da leitura de arquivos no momento do cálculo.
- Casos de Teste Mínimos: Garanta pelo menos um caso de teste para cada condição de reprovação e de aprovação de cada bolsa listada.
- Métrica JaCoCo: A classe `AnaliseElegibilidade` deve alcançar, no mínimo, 90% de cobertura de linhas e 90% de cobertura de branches.

Arquivo pom.xml

(Vocês devem atualizar o de vocês para resolver as dependências do JUnit e do Jacoco)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>br.edu.ufrgs</groupId>
  <artifactId>exemplo-servlet</artifactId>
  <version>1.0</version>
  <packaging>war</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- Servlet API -->
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>

    <!-- JUnit 5 (Jupiter) para os Testes -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <finalName>ROOT</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
      </plugin>

      <!-- Plugin do Maven Surefire (Obrigatório para rodar JUnit 5 no Maven) -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
    </plugins>
  </build>
</project>
```

```
<!-- Plugin do JaCoCo -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <!-- Inicializa o agente do JaCoCo antes dos testes rodarem -->
    <execution>
      <id>prepare-agent</id>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <!-- Gera o relatório HTML após a execução dos testes -->
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
</plugins>
</build>
</project>
```